

Welcome!

Nameplates please. And technology encouraged today!

All TF materials are available at github.com/nolankav/api-203.

If you want to follow along, download the dataset here:

In R: `df <- read.csv("http://tinyurl.com/api-202-tf-3")`

Parks and Prediction

API 203: TF Session 4

R

Nolan M. Kavanagh
April 3, 2026



Goals for today

- 1. Briefly touch on the principles of non-linear models.**
- 2. Contrast the goals of causal inference and prediction.**
- 3. Review a few strategies for predicting modeling, including LASSO.**
- 4. Learn how to generate predictive models in R.**
- 5. Practice evaluating the accuracy of predictive models.**

We'll treat this session like a workshop with an interactive example.

Overview of our sample data

Dataset of U.S. county-level characteristics in 2020

state	State of county	Administrative
county_fips	County FIPS identifier	Administrative
pc_under_18	Percent of county under age 18	American Community Survey (2016–2020)
pc_over_65	Percent of county over age 65	American Community Survey (2016–2020)
pc_male	Percent of county that is male	American Community Survey (2016–2020)
pc_black	Percent of county that is Black	American Community Survey (2016–2020)
pc_latin	Percent of county that is Hispanic/Latino	American Community Survey (2016–2020)
pc_hs_grad	Percent of county that graduated high school	American Community Survey (2016–2020)
unemploy_rate	County unemployment rate (%)	American Community Survey (2016–2020)
med_income_000s	County median income (\$1,000s)	American Community Survey (2016–2020)
pc_uninsured	Percent of county without health insurance	American Community Survey (2016–2020)
pc_trump	Percent of county votes for Trump in 2020	MIT Election Lab

A brief detour on non-linear models

- **Linear models are a good default, absent evidence of non-linearity.**
- **But when we want a non-linear model, we have several options.**
 - Higher-order polynomials: $Y = \beta_0 + \beta_1 * X + \beta_2 * X^2 + \dots$
 - Transformations to Y and/or X, such as logs: $\ln(Y) = \beta_0 + \beta_1 * X + \dots$
 - Others: percentiles/ranks, Winsorization
- **With non-linear models, we must be careful about interpretation.**

A brief detour on non-linear models

Model	Regression Equation (PRF)	Interpretation of β_1 (Math)	Interpretation of β_1 (Words)
Level-Level	$Y = \beta_0 + \beta_1(X_1) + u$	$\Delta Y = (\beta_1)\Delta X_1$	A 1 unit increase in X_1 is associated with a β_1 change in Y
Level-Log	$Y = \beta_0 + \beta_1(\ln X_1) + u$	$\Delta Y = (\frac{\beta_1}{100})\% \Delta X_1$	A 1% increase in X_1 is associated with a $0.01\beta_1$ change in Y
Log-Level	$\ln(Y) = \beta_0 + \beta_1(X_1) + u$	$\% \Delta Y = (100\beta_1)\Delta X_1$	A 1 unit increase in X_1 is associated with a $100\beta_1\%$ change in Y
Log-Log	$\ln(Y) = \beta_0 + \beta_1(\ln X_1) + u$	$\% \Delta Y = (\beta_1)\% \Delta X_1$	A 1% increase in X_1 is associated with a $\beta_1\%$ change in Y

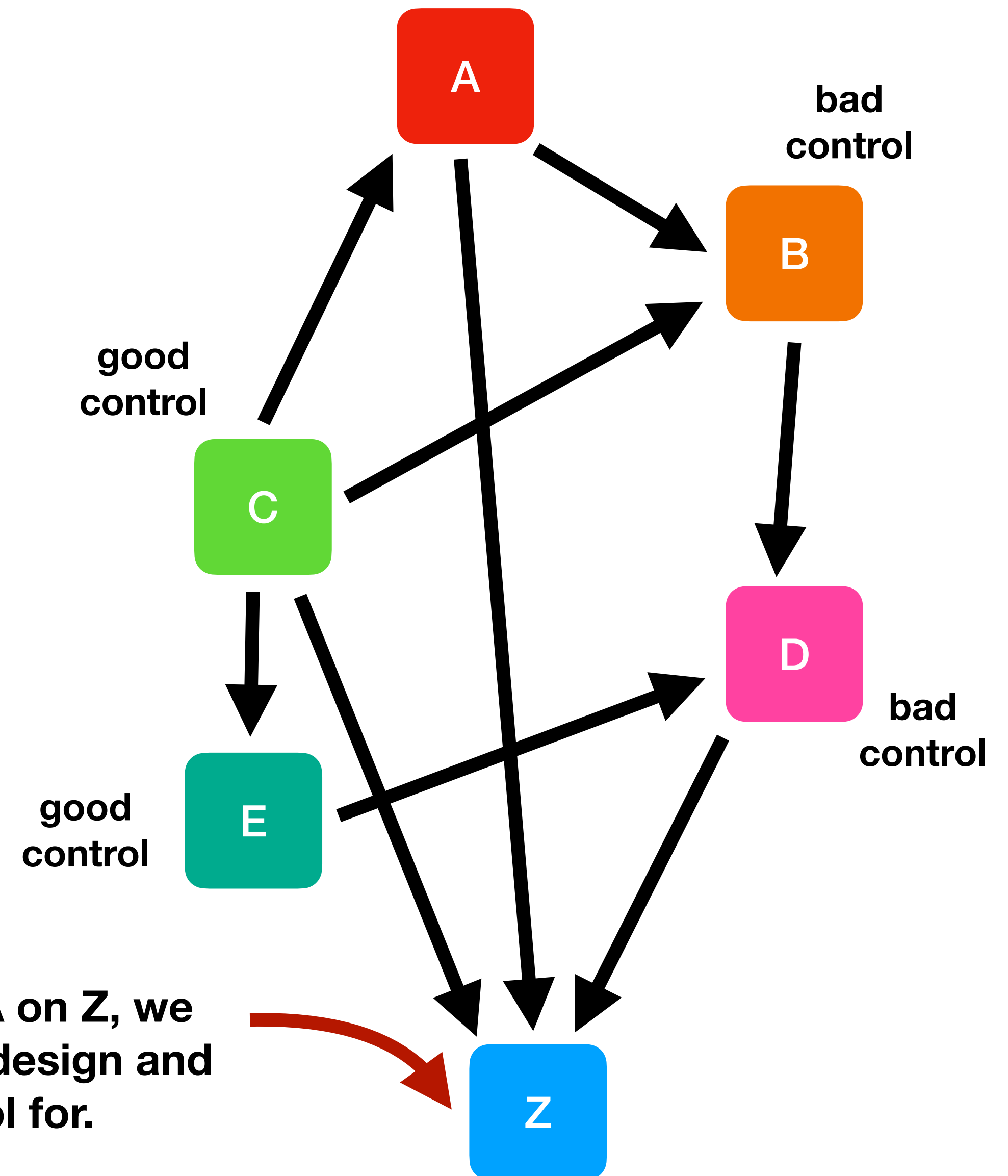
Finally, the “percent” interpretation we are often tempted to make with non-transformed linear models!

What's our goal?

With causal inference, the name of the game was bias: what causes it and how to get rid of it.

Why? Our goal was to estimate an unbiased effect.

To get the causal estimate of A on Z, we have to be careful about study design and which variables we control for.



What's our goal?

With causal inference, the name of the game was bias: what causes it and how to get rid of it.

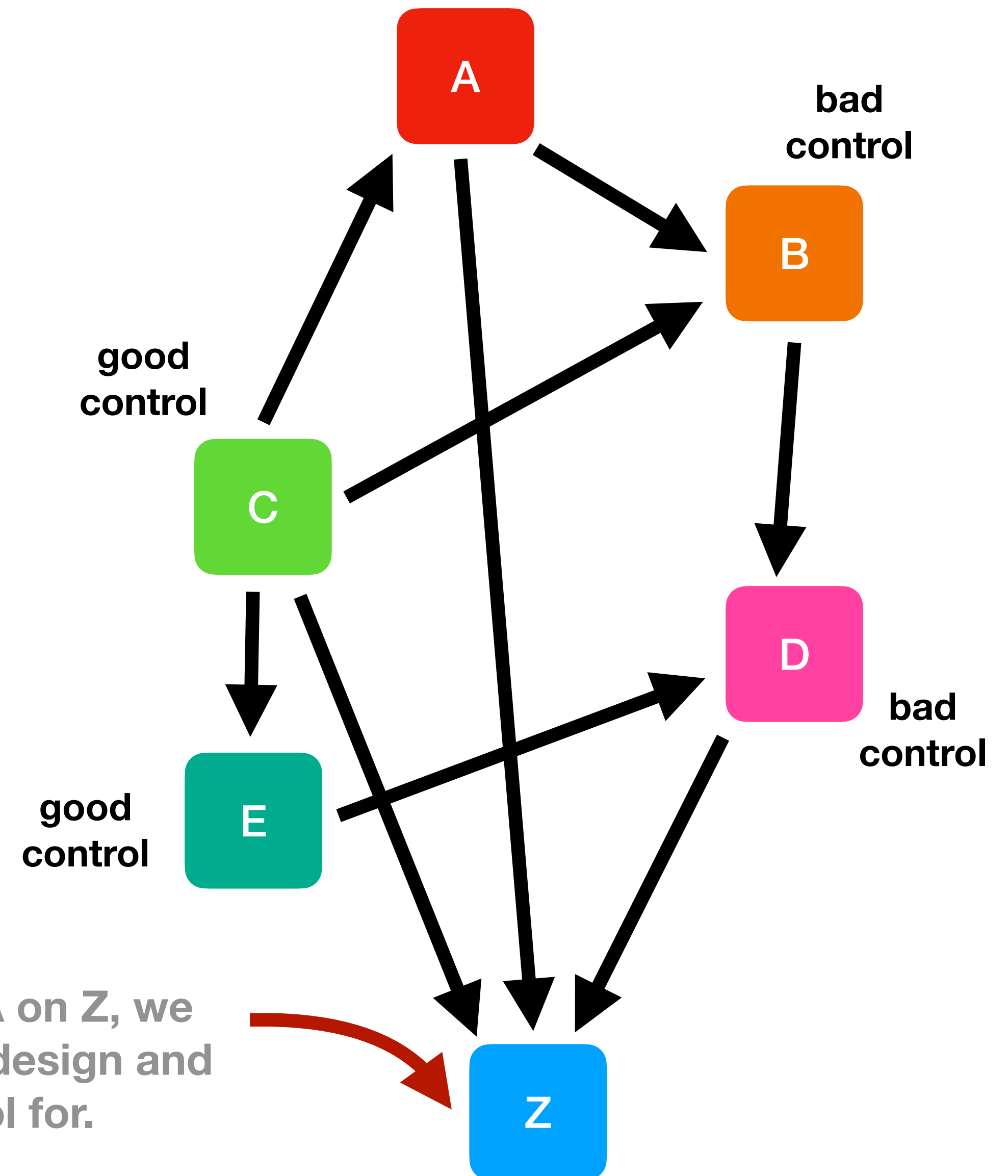
Why? Our goal was to estimate an unbiased effect.

With prediction, we don't care about bias. In fact, we invite bias. We'll use whatever we can.

Why? Our goal is accurate prediction.

To get the causal estimate of A on Z, we have to be careful about study design and which variables we control for.

But if we just want to predict Z, every relevant variable is fair game.



What's our goal?

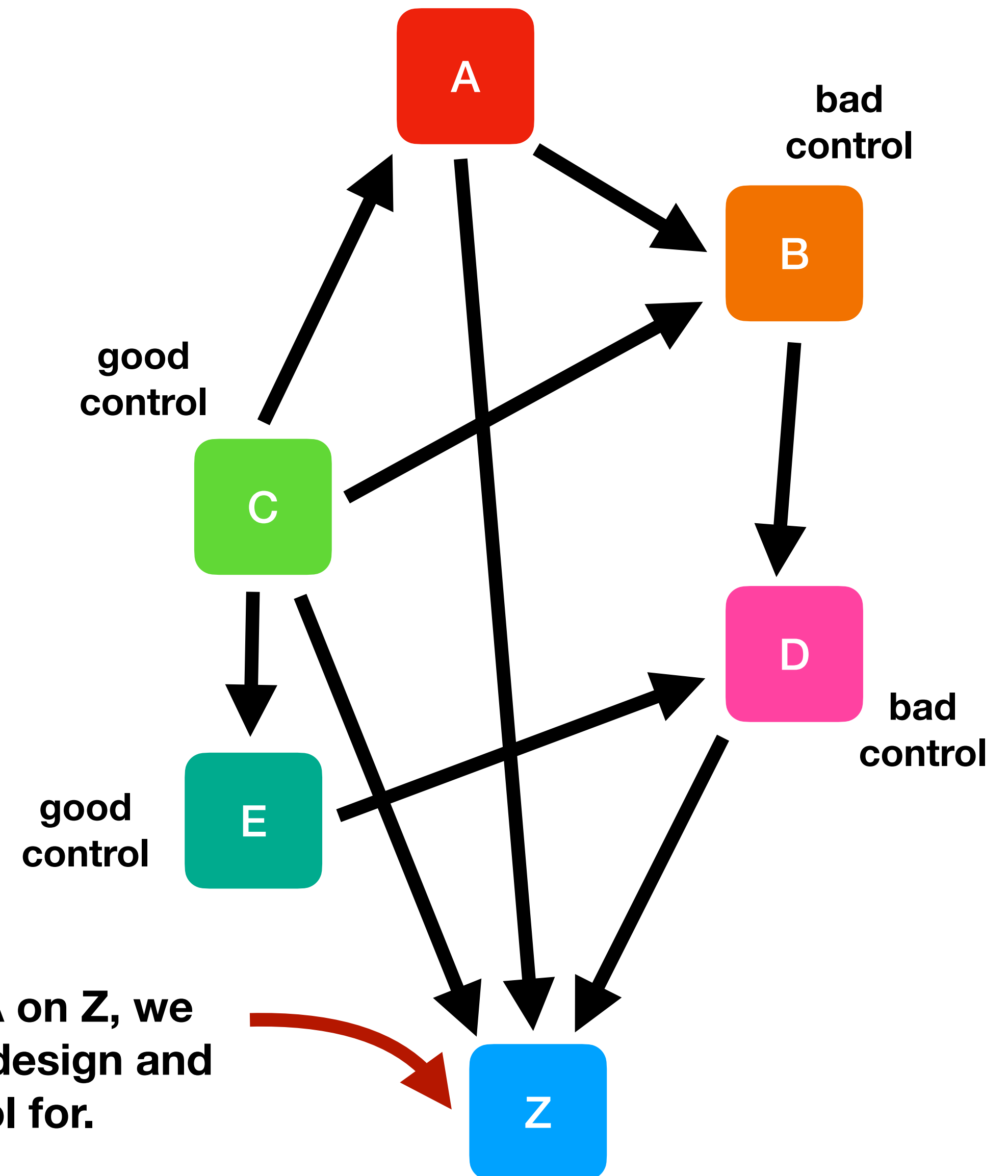
With all possible variables at our disposal, we do have to worry about overfitting our models.

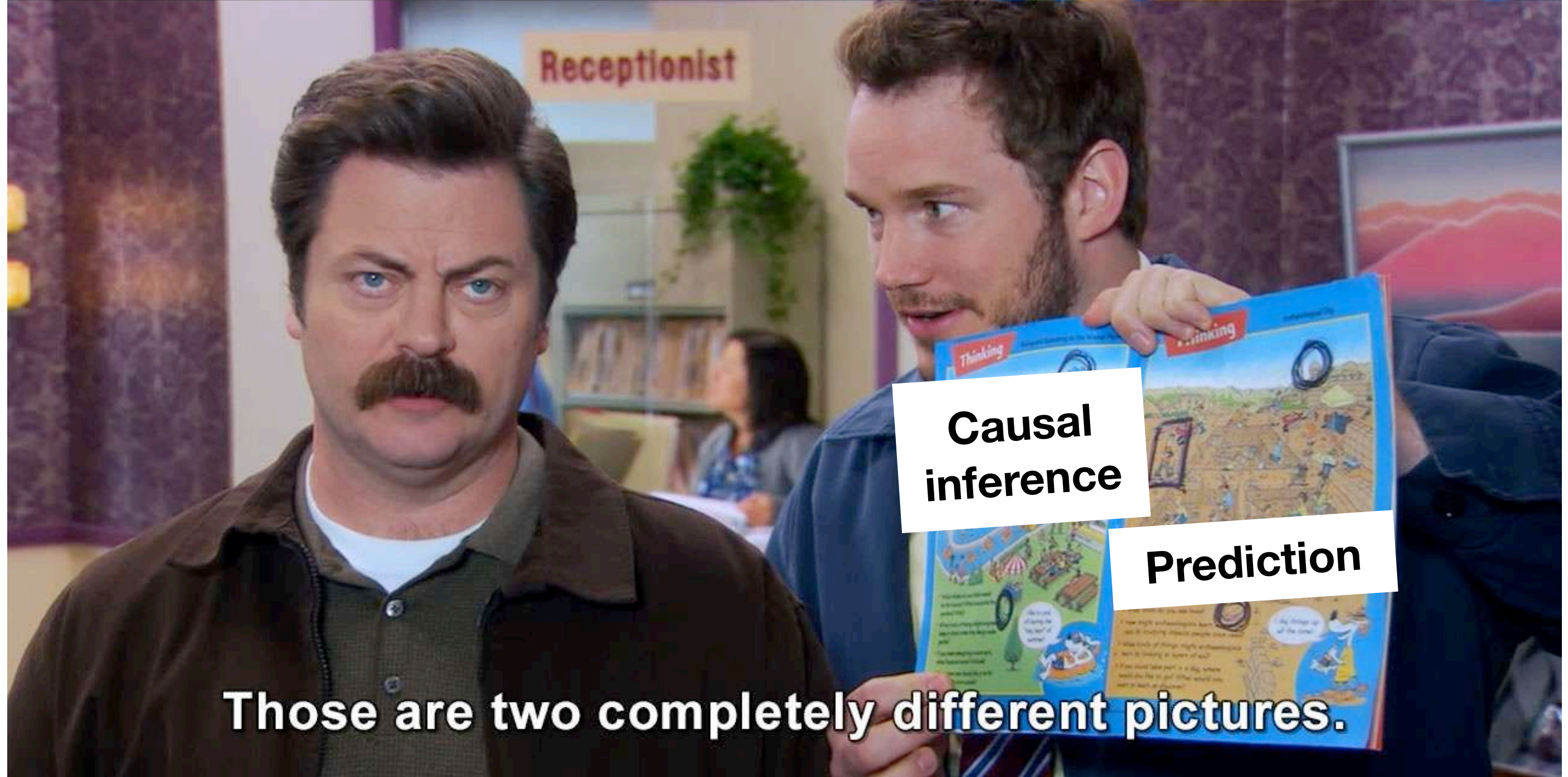
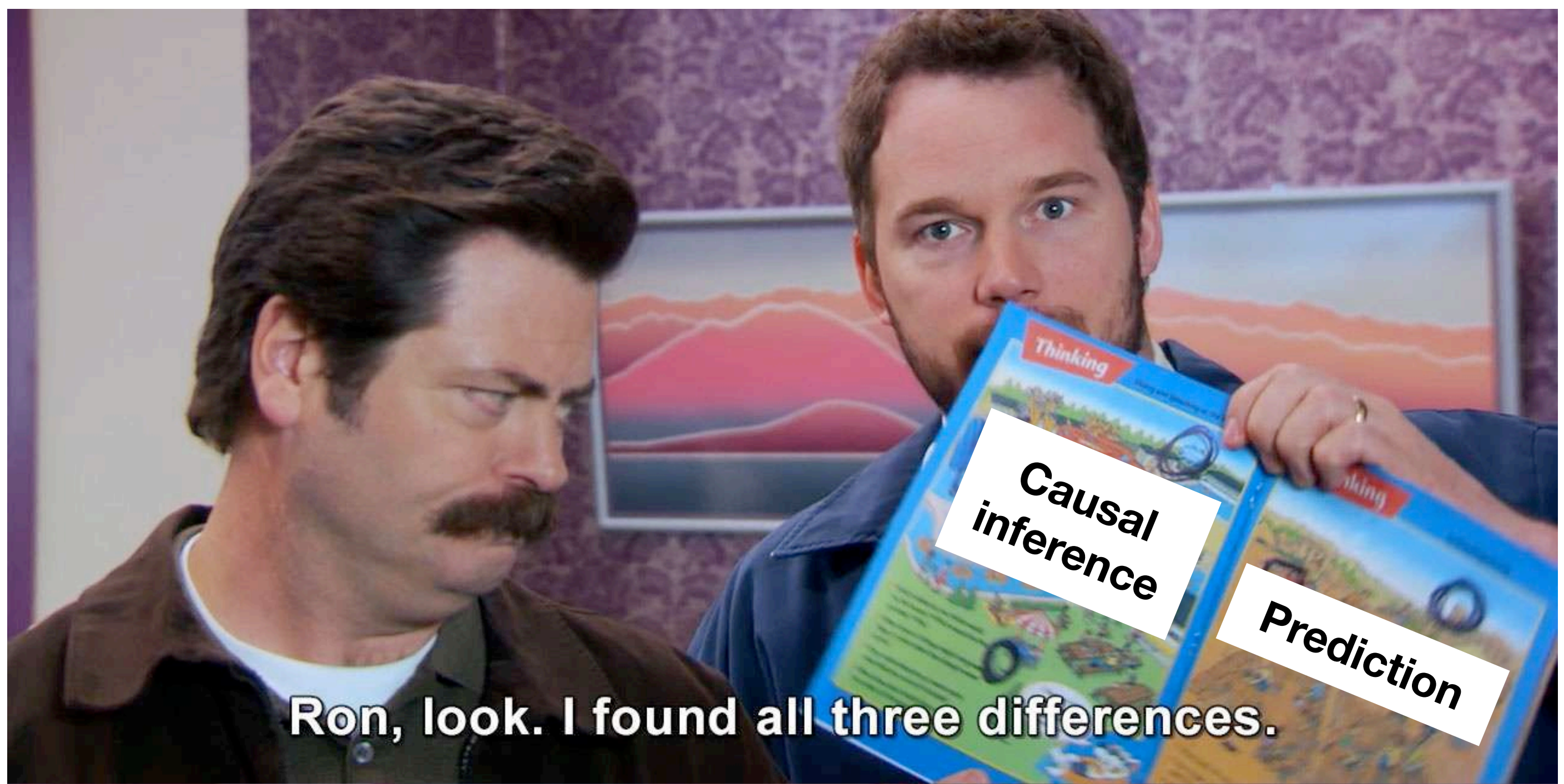
Overfitted models look good in-sample, but they tend to perform poorly out-of-sample.

Above all, we want useful predictions!

To get the causal estimate of A on Z, we have to be careful about study design and which variables we control for.

But if we just want to predict Z, every relevant variable is fair game.





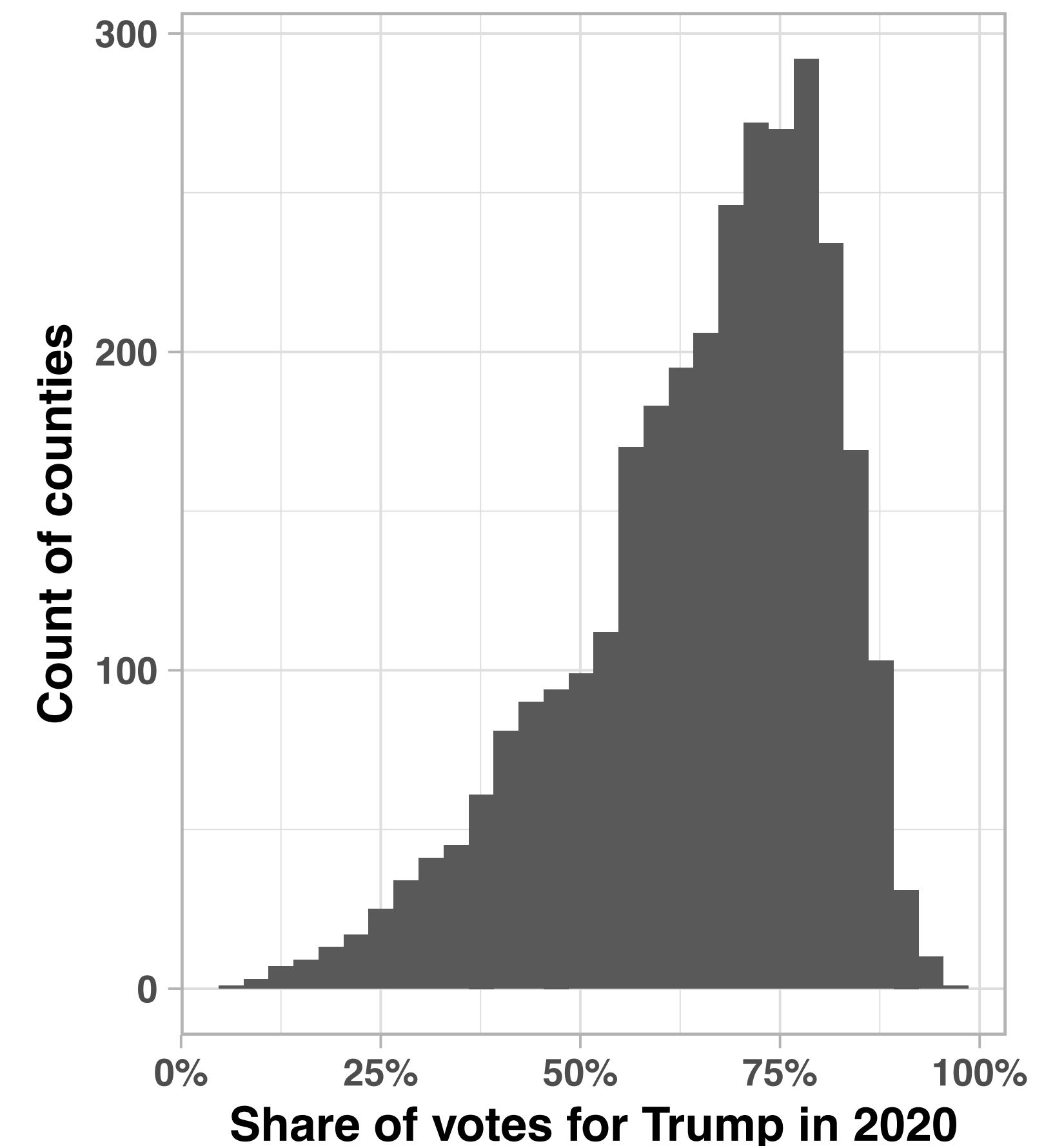
OK, so what are we predicting?

Let's try to predict election outcomes.

This is what prediction sites do publicly (and what political parties do privately).

We're doing “supervised” machine learning since we can validate our predictions on the “truth.”

```
# Plot distribution of Republican support
plot_1 <- ggplot(data=df, aes(x=pc_trump)) +
  geom_histogram() +
  xlab("Share of votes for Trump in 2020") +
  ylab("Count of counties") +
  theme_light() +
  theme(text = element_text(size = 10, face = "bold")) +
  scale_x_continuous(labels = function(x) paste0(x,"%"))
```



First, let's split our data into two.

We want to designate “training” and “test” sets. One is for developing our model, and the other is for validating its predictions.

Here, we will do a 70/30 split. Generally, we want more data in the training set.

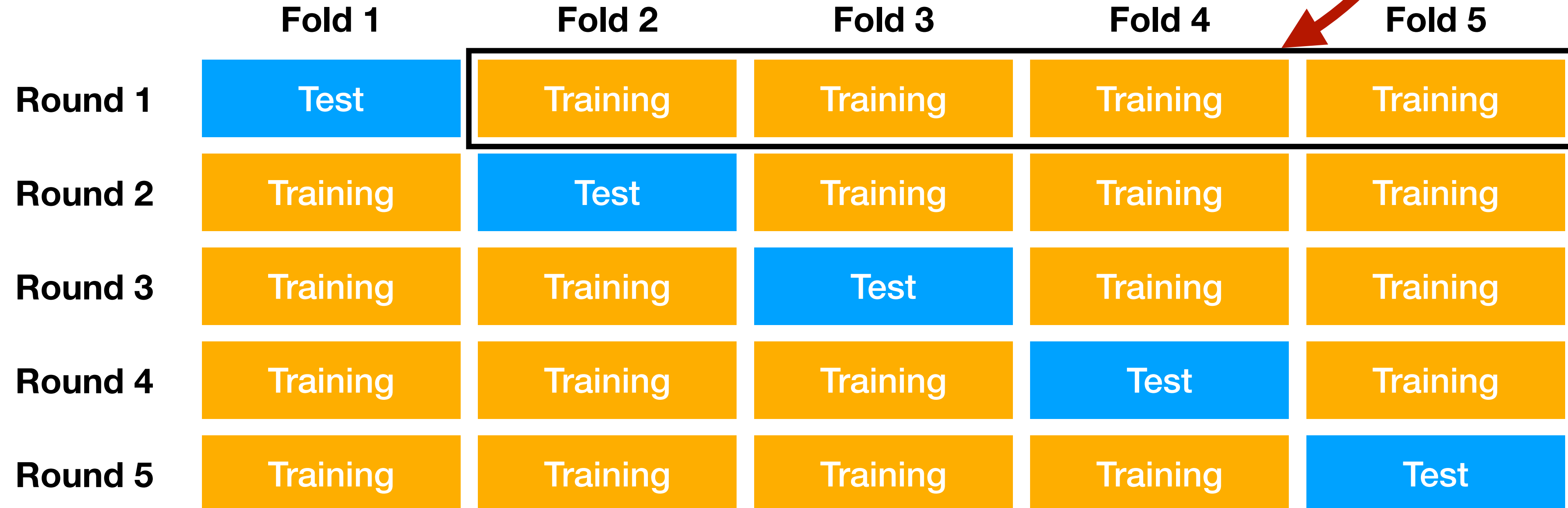
```
# Designate training and test sets
set.seed(1234)
split      <- initial_split(df, 0.7)
df_train   <- training(split)
df_test    <- testing(split)
```

An advanced approach is K-fold cross-validation.

This approach splits the data into K equal folds.

Then, we use each fold as the test set for one round of modeling and average across all rounds.

In each round, we pool all the training folds into one big training set.



	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Round 1	Test	Training	Training	Training	Training
Round 2	Training	Test	Training	Training	Training
Round 3	Training	Training	Test	Training	Training
Round 4	Training	Training	Training	Test	Training
Round 5	Training	Training	Training	Training	Test

A photograph of a man and a woman sitting at a wooden table, each holding a glass of red wine. The man, on the left, is wearing a dark suit and tie, looking towards the woman. The woman, on the right, is wearing a grey and white striped sweater with a colorful orange and yellow scarf, smiling at the man. The background shows a wooden door and a blue wall. Three white text boxes are overlaid on the image: 'Predictive modeling' on the man, 'Ruining springtime for MPPs' in the center, and 'Causal inference' on the woman.

Predictive modeling

**Ruining springtime
for MPPs**

Causal inference

Let's train a very basic model.

```
# OLS regression
```

```
model_1 <- lm(pc_trump ~ med_inc_000s, data=df_train)
summary(model_1)
```

Call:

```
lm(formula = pc_trump ~ med_inc_000s, data = df_train)
```

Residuals:

Min	1Q	Median	3Q	Max
-63.017	-9.027	3.203	11.103	34.338

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	81.85852	1.25994	64.97	<2e-16 ***
med_inc_000s	-0.30425	0.02215	-13.73	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 15.36 on 2177 degrees of freedom

Multiple R-squared: 0.07973, Adjusted R-squared: 0.07931

F-statistic: 188.6 on 1 and 2177 DF, p-value: < 2.2e-16

Let's train a very basic model.

```
# OLS regression
model_1 <- lm(pc_trump ~ med_inc_000s, data=df_train)
summary(model_1)

Call:
lm(formula = pc_trump ~ med_inc_000s, data = df_train)

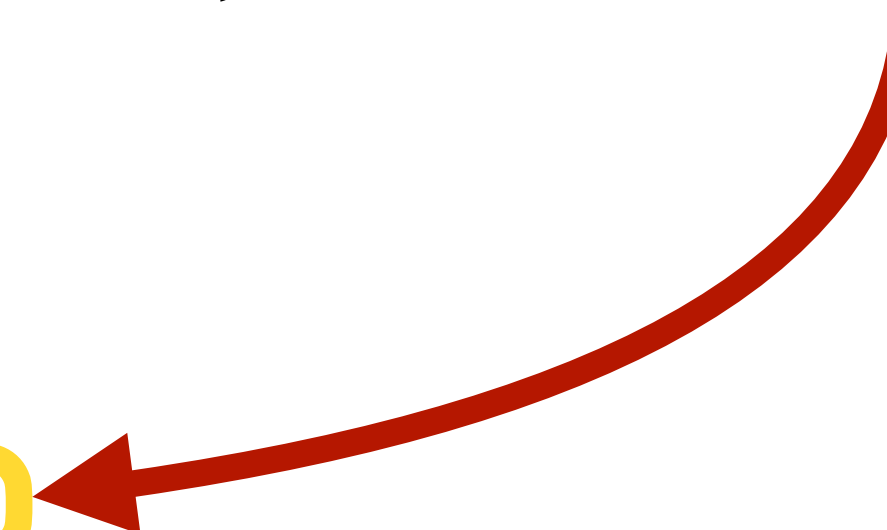
Residuals:
    Min       1Q   Median       3Q      Max
-63.017  -9.027   3.203  11.103  34.338

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  81.85852    1.25994   64.97  <2e-16 ***
med_inc_000s -0.30425    0.02215  -13.73  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 15.36 on 2177 degrees of freedom
Multiple R-squared:  0.07973,    Adjusted R-squared:  0.07931
F-statistic: 188.6 on 1 and 2177 DF,  p-value: < 2.2e-16
```

One way to evaluate predictive models is R-squared. It's less ideal than root mean square error (RMSE), which is measured in units of our outcome, but R-squared can be useful at a glance.

R2 = 8%, which isn't super high...



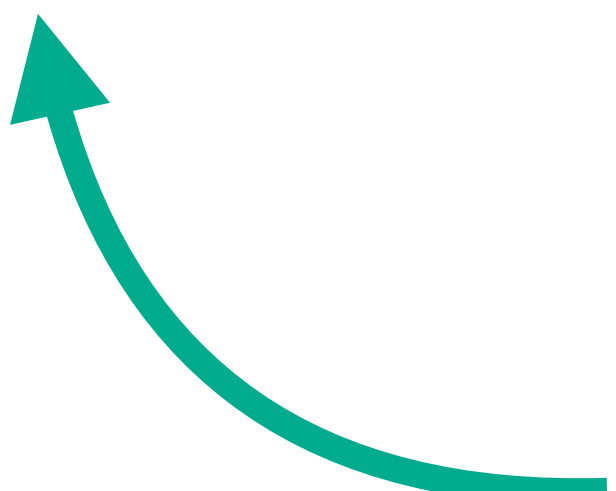
Let's evaluate our predictions!

```
# In-sample prediction & MSE
df_train$predict <- predict(model_1, df_train)
df_train %>% summarise(mse = mean((pc_trump - predict)^2))
```

```
      mse
1 235.6529
```

```
# Out-of-sample prediction & MSE
df_test$predict <- predict(model_1, df_test)
df_test %>% summarise(mse = mean((pc_trump - predict)^2))
```

```
      mse
1 250.4202
```



To compute the MSE, we take the error between the true and predicted values, square them, and take the mean.

Hence, mean squared error.

Let's evaluate our predictions!

```
# In-sample prediction & MSE
```

```
df_train$predict <- predict(model_1, df_train)
df_train %>% summarise(mse = mean((pc_trump - predict)^2))
```

```
      mse
1 235.6529
```

```
# Out-of-sample prediction & MSE
```

```
df_test$predict <- predict(model_1, df_test)
df_test %>% summarise(mse = mean((pc_trump - predict)^2))
```

```
      mse
1 250.4202
```

To get the RMSE, we just take the square root:

$\text{sqr}(235.6) = 15.4 \text{ pp}$

$\text{sqr}(250.4) = 15.8 \text{ pp}$

Remember that our scale is 0 to 100%, so on average, we are off by a fair amount!

Also, notice that the in-sample predictions are a tiny bit better than out-of-sample predictions.

Let's train a more complicated model.

```
# OLS regression
model_2 <- lm(pc_trump ~ med_inc_000s + pc_hs_grad + pc_male + pc_latin, data=df_train)
summary(model_2)
```

Call:

```
lm(formula = pc_trump ~ med_inc_000s + pc_hs_grad + pc_male +
    pc_latin, data = df_train)
```

Residuals:

Min	1Q	Median	3Q	Max
-57.111	-8.325	0.971	9.325	40.863

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	94.92910	7.38851	12.848	< 2e-16	***
med_inc_000s	-0.04116	0.02454	-1.677	0.0937	.
pc_hs_grad	-0.92603	0.05192	-17.834	< 2e-16	***
pc_male	0.68146	0.12501	5.451	5.57e-08	***
pc_latin	-0.05175	0.02254	-2.296	0.0218	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.98 on 2174 degrees of freedom

Multiple R-squared: 0.2389, Adjusted R-squared: 0.2375

F-statistic: 170.6 on 4 and 2174 DF, p-value: < 2.2e-16

Let's train a more complicated model.

```
# OLS regression
model_2 <- lm(pc_trump ~ med_inc_000s + pc_hs_grad + pc_male + pc_latin, data=df_train)
summary(model_2)
```

Call:

```
lm(formula = pc_trump ~ med_inc_000s + pc_hs_grad + pc_male +
    pc_latin, data = df_train)
```

Residuals:

Min	1Q	Median	3Q	Max
-57.111	-8.325	0.971	9.325	40.863

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	94.92910	7.38851	12.848	< 2e-16	***
med_inc_000s	-0.04116	0.02454	-1.677	0.0937	.
pc_hs_grad	-0.92603	0.05192	-17.834	< 2e-16	***
pc_male	0.68146	0.12501	5.451	5.57e-08	***
pc_latin	-0.05175	0.02254	-2.296	0.0218	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.98 on 2174 degrees of freedom

Multiple R-squared: 0.2389, Adjusted R-squared: 0.2375

F-statistic: 170.6 on 4 and 2174 DF, p-value: < 2.2e-16

The R-squared went up: 7% vs. 24%.
That's a good sign!

Let's evaluate our predictions!

```
# In-sample prediction & MSE
df_train$predict_2 <- predict(model_2, df_train)
df_train %>% summarise(mse = mean((pc_trump - predict_2)^2))
```

```
      mse
1 194.9048
```

```
# Out-of-sample prediction & MSE
df_test$predict_2 <- predict(model_2, df_test)
df_test %>% summarise(mse = mean((pc_trump - predict_2)^2))
```

```
      mse
1 199.4195
```

Let's evaluate our predictions!

```
# In-sample prediction & MSE
df_train$predict_2 <- predict(model_2, df_train)
df_train %>% summarise(mse = mean((pc_trump - predict_2)^2))
```

```
      mse
1 194.9048
```

```
# Out-of-sample prediction & MSE
df_test$predict_2 <- predict(model_2, df_test)
df_test %>% summarise(mse = mean((pc_trump - predict_2)^2))
```

```
      mse
1 199.4195
```

To get the RMSE, we take the square root:

$\text{sqr}(194.9) = 14.0 \text{ pp}$

$\text{sqr}(199.4) = 14.1 \text{ pp}$

This is better than our last model!



**Making predictions with
random models.**

“I have no idea what I’m doing,”



**Making predictions with
random models.**

“But I know I’m doing it well”

Let's bring out the big guns.

We'll use every variable and interact them!

```
# OLS regression
model_3 <- lm(pc_trump ~
med_inc_000s*pc_hs_grad*unemploy_rate*pc_uninsured*pc_under_18*
pc_over_65*pc_male*pc_black*pc_latin, data=df_train)
```

```
# In-sample prediction & MSE
df_train$predict_3 <- predict(model_3, df_train)
df_train %>% summarise(mse = mean((pc_trump - predict_3)^2))
```

```
      mse
1 44.50534
```

```
# Out-of-sample prediction & MSE
df_test$predict_3 <- predict(model_3, df_test)
df_test %>% summarise(mse = mean((pc_trump - predict_3)^2))
```

```
      mse
1 1595.704
```

It doesn't matter whether we have a clear causal model that connects each of these variables to support for Trump.

Our goal is predictive accuracy, not causal identification!

Let's bring out the big guns.

We'll use every variable and interact them!

```
# OLS regression
model_3 <- lm(pc_trump ~
med_inc_000s*pc_hs_grad*unemploy_rate*pc_uninsured*pc_under_18*
pc_over_65*pc_male*pc_black*pc_latin, data=df_train)
```

```
# In-sample prediction & MSE
df_train$predict_3 <- predict(model_3, df_train)
df_train %>% summarise(mse = mean((pc_trump - predict_3)^2))
```

```
      mse
1 44.50534
```

```
# Out-of-sample prediction & MSE
df_test$predict_3 <- predict(model_3, df_test)
df_test %>% summarise(mse = mean((pc_trump - predict_3)^2))
```

```
      mse
1 1595.704
```

Our in-sample predictions got better...
 $\text{sqr}(44.5) = 6.7$ pp

...but out-of-sample, they're terrible!
 $\text{sqr}(1595.7) = 39.9$ pp

This is what happens when we “overfit” our model.
We have only 2,179 counties in the training set yet fit 512 coefficients!

We need some guardrails.

Enter: LASSO regression.

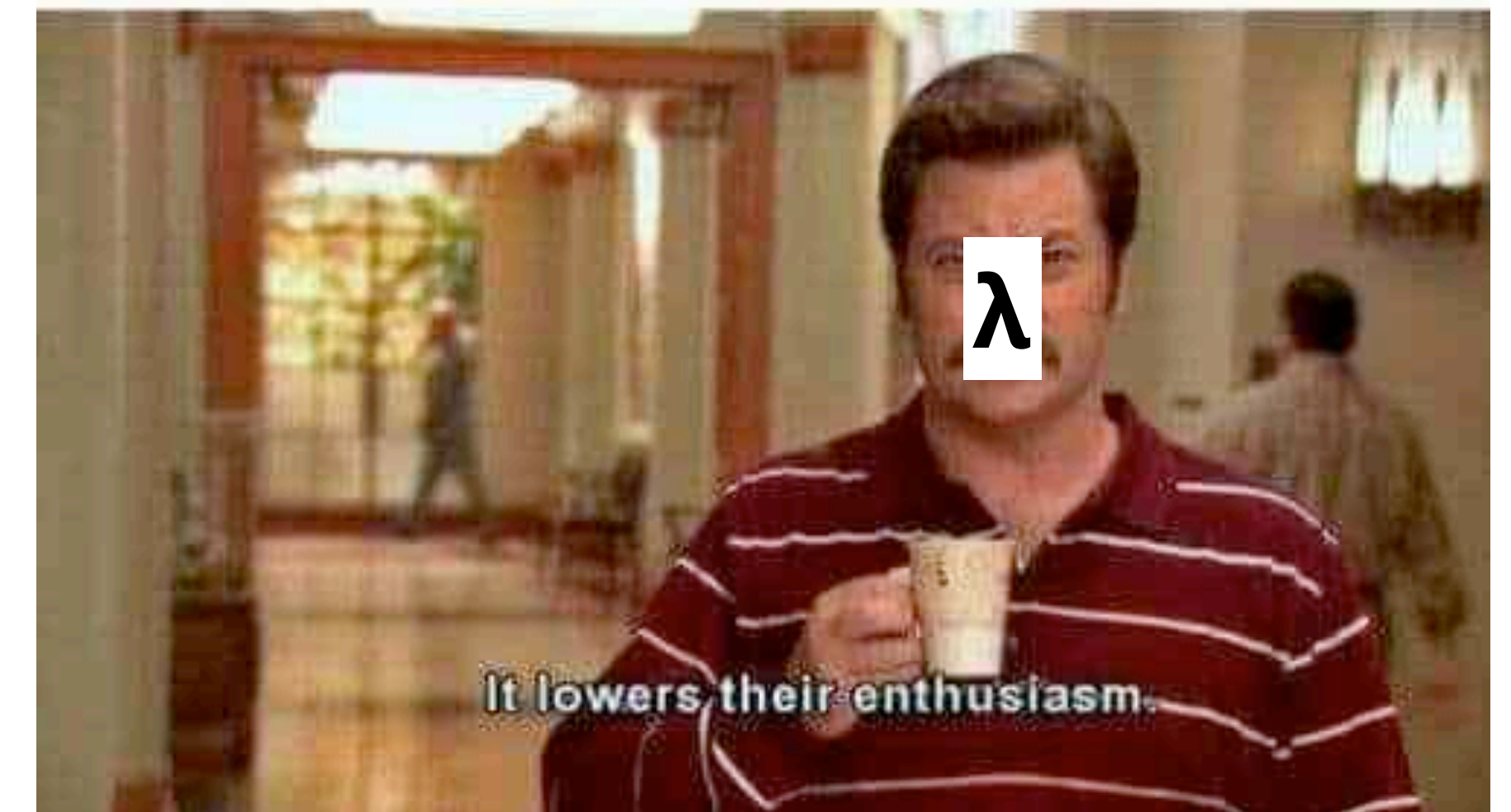
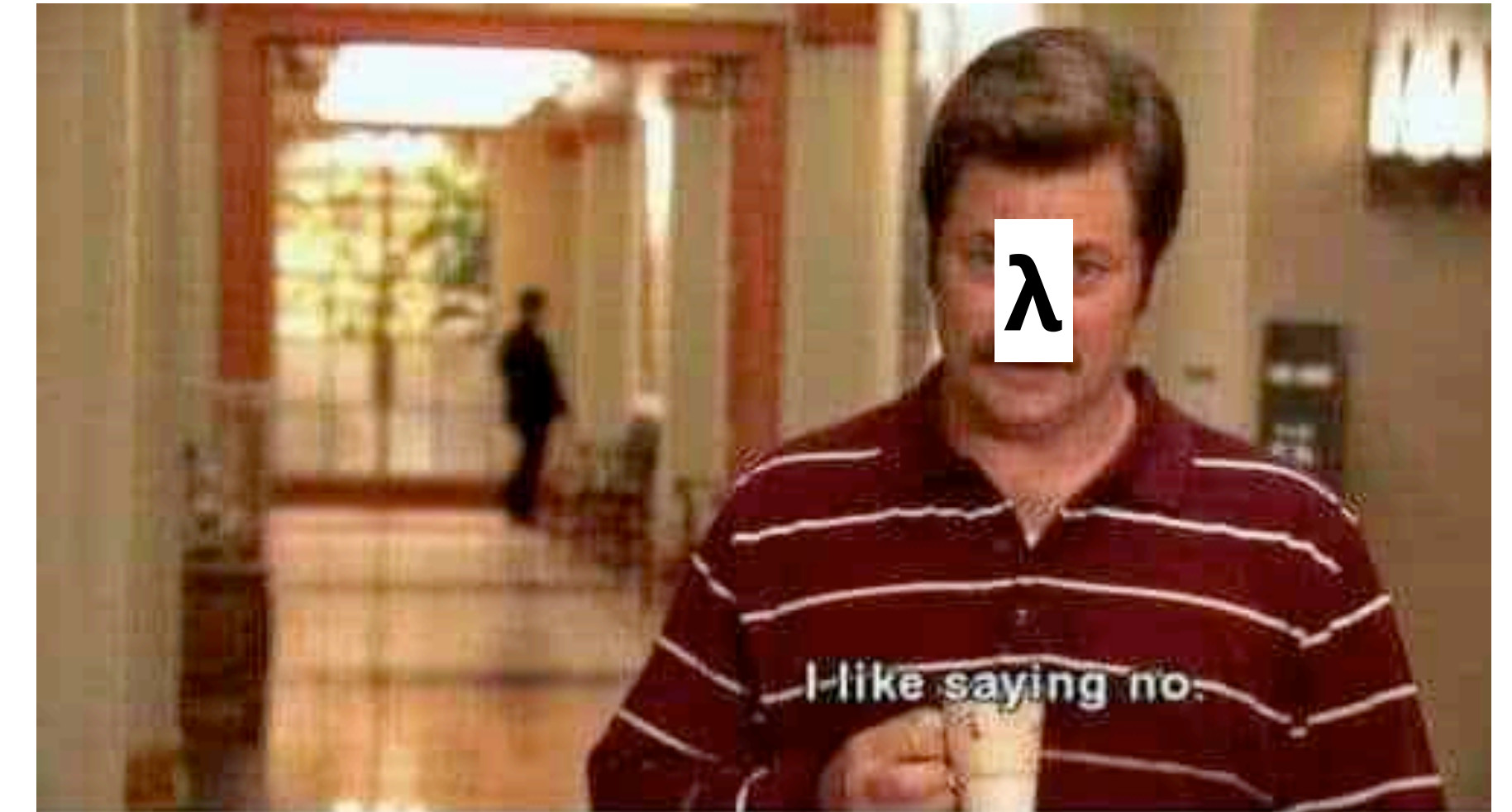
This is normal OLS.

$$\hat{\beta}_{\lambda}^{LASSO} = \arg \min_{\beta} \left[\underbrace{\sum_{i=1}^n \left(y_i - (\beta_0 + \sum_{j=1}^p \beta_j x_{ij}) \right)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=1}^p |\beta_j|}_{\text{Regularizer}} \right]$$

Where $\lambda \geq 0$ is called a *tuning parameter*.

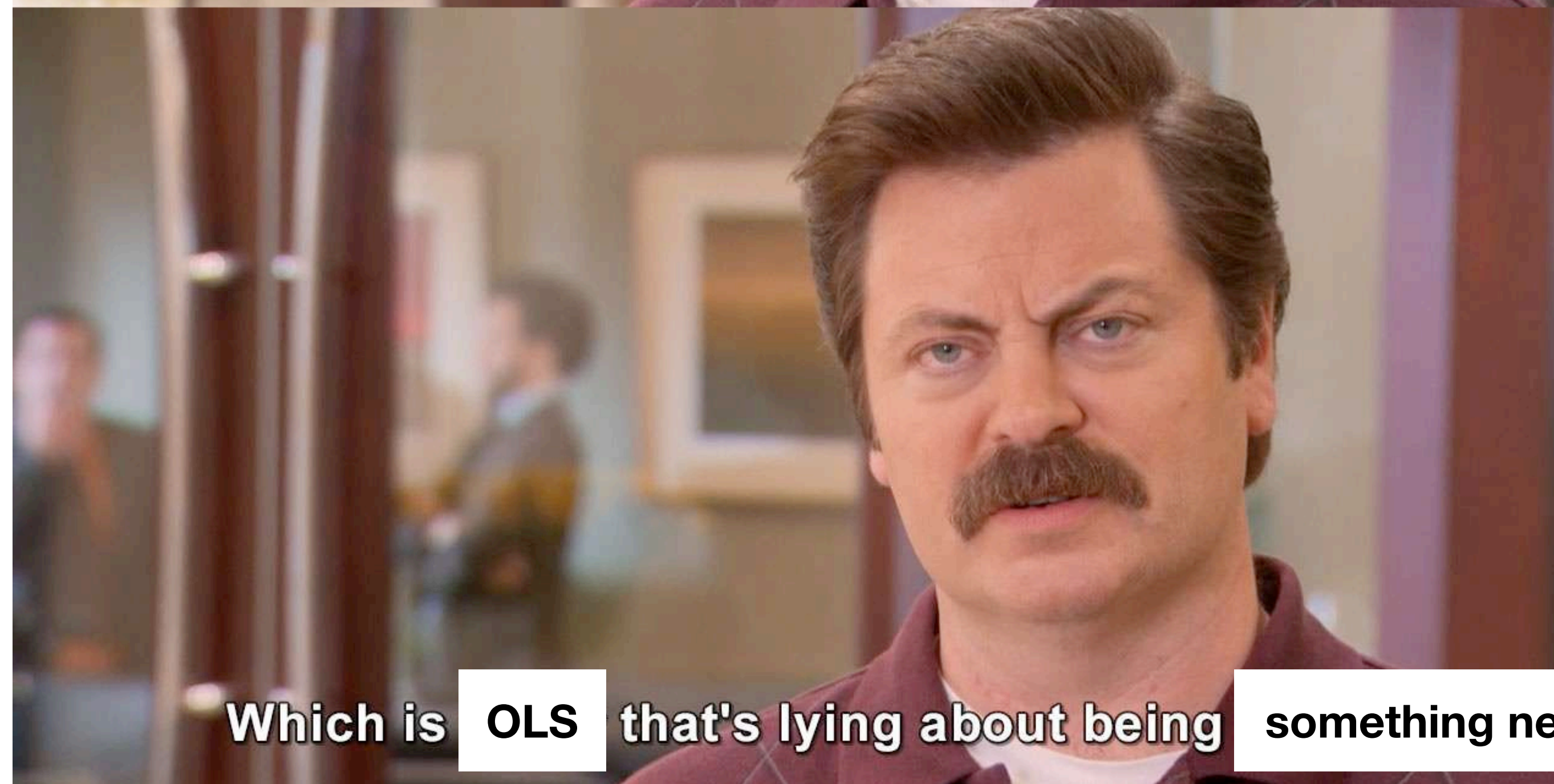
This penalizes us for more and larger betas.

This way, we reach a point at which adding more betas doesn't improve fit.





There's only one thing I hate more than **OLS**, **LASSO** .



Which is **OLS** that's lying about being something new and fancy

Alright, here we go.

First, we will load the glmnet library.

```
# Load library  
library(glmnet)
```

Second, we will select the Y and X variables that we want to use.

```
# Set the variables that are "fair game"  
Y <- df_train$pc_trump  
X <- data.matrix(df_train[, c("med_inc_000s", "pc_hs_grad", "unemploy_rate", "pc_uninsured",  
                             "pc_under_18", "pc_over_65", "pc_male", "pc_black", "pc_latin")])
```

```
# LASSO regression  
lasso <- cv.glmnet(x=X, y=Y)  
plot(lasso); log(lasso$lambda.min)
```

Third, we run the model.

```
[1] -3.735557
```

Lastly, we will evaluate the lambda that was selected.

Alright, here we go.

```
# Load library
library(glmnet)

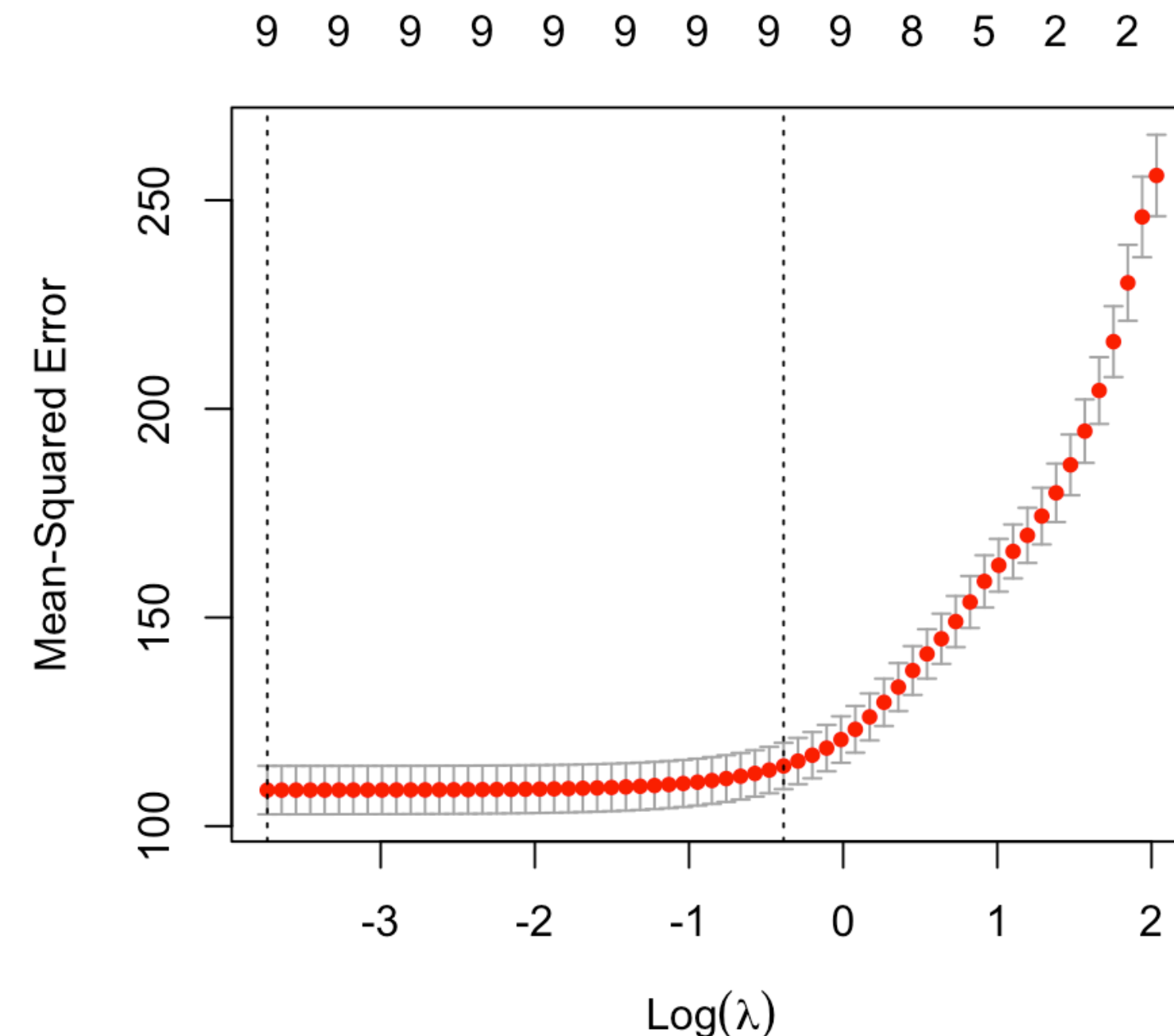
# Set the variables that are "fair game"
Y <- df_train$pc_trump
X <- data.matrix(df_train[, c("med_inc_000s", "pc_hs_grad", "unemploy_rate", "pc_uninsured",
                              "pc_under_18", "pc_over_65", "pc_male", "pc_black", "pc_latin")])

# LASSO regression
lasso <- cv.glmnet(x=X, y=Y)
plot(lasso); log(lasso$lambda.min)

[1] -3.735557
```

Here's all the $\log(\lambda)$ s plotted.

We want the minimum lambda, as this value minimizes the MSE.



Show me the predictions already!

Be sure to use the minimum lambda!



```
# In-sample prediction & MSE
```

```
df_train$predict_4 <- predict(lasso, newx=X, s="lambda.min")[, 1]  
df_train %>% summarise(mse = mean((pc_trump - predict_4)^2))
```

```
      mse  
1 107.2424
```

```
# Get X variables of test set
```

```
X_test <- data.matrix(df_test[, c("med_inc_000s", "pc_hs_grad", "unemploy_rate", "pc_uninsured",  
                                "pc_under_18", "pc_over_65", "pc_male", "pc_black", "pc_latin")])
```

```
# Out-of-sample prediction & MSE
```

```
df_test$predict_4 <- predict(lasso, newx=X_test, s="lambda.min")[, 1]  
df_test %>% summarise(mse = mean((pc_trump - predict_4)^2))
```

```
      mse  
1 109.7354
```

Show me the predictions already!

```
# In-sample prediction & MSE
```

```
df_train$predict_4 <- predict(lasso, newx=X, s="lambda.min")[, 1]  
df_train %>% summarise(mse = mean((pc_trump - predict_4)^2))
```

```
      mse  
1 107.2424
```



In-sample RMSE: $\sqrt{107.2} = 10.4$ pp

```
# Get X variables of test set
```

```
X_test <- data.matrix(df_test[, c("med_inc_000s", "pc_hs_grad", "unemploy_rate", "pc_uninsured",  
                                "pc_under_18", "pc_over_65", "pc_male", "pc_black",  
                                "pc_latin")])
```

```
# Out-of-sample prediction & MSE
```

```
df_test$predict_4 <- predict(lasso, newx=X_test, s="lambda.min")[, 1]  
df_test %>% summarise(mse = mean((pc_trump - predict_4)^2))
```

```
      mse  
1 109.7354
```



Out-of-sample RMSE: $\sqrt{109.7} = 10.5$ pp

So far, this is our best model! That said, performance will vary by context and available data.



LASSO

OLS

loves it when you can show it you're better than it
is at something it loves

Let's compare our models.

	In-sample RMSE	Out-of-sample RMSE
OLS (income)	15.4 pp	15.8 pp
OLS (income, education, gender, Latin)	14.0 pp	14.1 pp
OLS (all variables interacted)	6.7 pp	39.9 pp
LASSO (all variables additive)	10.4 pp	10.5 pp

Of the models we tried, the LASSO one performs best.

Is there a better way...?



Predictions, Part 1.

**Remember to keep your eye on the prize:
It's all about predictive accuracy.**

Be careful not to overfit the model.

**LASSO regression can help reduce
overfitting by penalizing extra terms.**

But there are even better ways...

